RnD Report

Adithya
Supervised by: Prof. Pushpak Bhattacharyya,
Computer Science and Engineering,
Indian Institute of Technology Bombay.
adithya@cse.iitb.ac.in

April 19, 2016

Contents

Chapter 1

Introduction

Automatic Summarization is an automated way (by a computer program) of generate a summary of an input document which briefly describes the document by presenting the most important and relevant aspects of the document. This is a widely studied problem and comes broadly under the domain of machine learning and natural language processing.

Two main approaches to the task of automatic summarization are extraction-based and abstraction-based. Extractive summarization consists of directly extracting the objects (possibly sentences or words) from the document without any changes to the sentence structure itself. In contrary, abstractive summarization paraphrases the units of document to generate a much more condensed summary. Abstractive summarization requires the techniques of natural language generation. Though most of the work in this domain has been done via extraction-based approach, the research in the abstractive domain has become increasingly important. It closely resembles human way of generating the summary and is an active area of research.

This RnD work is focused mainly on learning of the state of art techniques of automatic summarization over the last couple of decades and developing a working model of an extractive summarization technique. The initial chapters of this document consists of a short description of concepts discussed in various papers published in the area of Automatic Summarization, followed by Sentiment Analysis, Submodularity for Summarization and concludes by describing the working of a TextRank implementation.

Chapter 2

Automatic Summarization

2.1 TextRank

Based on readings of *TextRank: Bringing Order into Texts* [1].

Introduction: Develops a novel unsupervised learning method with proved applications both in keyword extraction and sentence extraction. Some other notable graph-based ranking algorithms are Kleinberg's HITS algorithm and Google's PageRank. Builds a graph on the content of text-based problem at hand and then determines the score for each vertex iteratively. One of the notable earlier works is a supervised learning system to keyword extraction from abstracts, using a combination of lexical and syntactic features. They compare the performance with above algorithm for both undirected and directed, weighted and unweighted graphs

Algorithm: Represent text units as vertices in the graph, identify the relations between these text units. Iterate through the graph until the error converges to a value below a pre-decided threshold. Co-occurence and sentence similarity are used as relational measures in the applications of keyword and summary extraction respectively. Number of keywords are chosen based on the size of the text. In the post processing stage, sequence of adjacent keywords are collapsed into a multi-word keyword.

$$WS(V_i) = (1-d) + d * \sum_{V_j \in In(V_i)} \frac{w_{ji}}{\sum_{V_k \in Out(V_j)} w_{jk}} WS(V_j) \tag{2.1}$$

Forming of web connections based on related text units is very similar to the way humans build summaries. The working system associated with this work is actually an implementation of TextRank both for extracting keywords and generating summary. It works on input Hindi texts and the complete procedure is explained in chapter 5

Results: TextRank has a better precision and f-measure but not recall due to the its limitation on the number of keywords selected. Since it is a unsupervised learning technique, it is easily adaptable to other languages and domains.

2.2 SUMMARIST

Based on readings of *Automatic Text Summarization in SUMMARIST* [2].

Key equation: *Summarization : Topic Identification → Interpretation → Generation*

Extract summaries consists wholly of portions directly extracted from original, but abstract summaries have novel phrasings describing the original content using natural language generation, topic fusion. Goal is to provide both abstract and extract summaries for various languages. IR- techniques are used in automatic summarization but abstract summaries require analysis and interpretation at levels deeper than word-level (ex: synonymy, polysemy). To employ IR techniques as far as they can take us, and then augment them with symbolic/semantic methods. Pre-processing: tokenizer, parts of speech tagger, token frequency counter are few component modules. Rating or a score is added to each of the tokens for further processing.

Topic Identification: Filtering the input to retain only the most important central topics. Two factors mainly influence this, quantity (more or less topics) and user's interest. Various submodules in this are position, cue phrases ("in summary", "in conclusion"), concept signatures (head, keyword pairs) and an integration module.

Topic Interpretation: In this stage, two or more extracted topics are fused into one or more unifying concepts. For example, drive in + pay + fill tank + close tank → gasoline fill up. These methods associate a set of concepts (indicators) with characteristic generalization (fuser, head). The goal is to build a large collection of these fuser-indicators sets. This includes Concept Wavefront and Concept Signature.

Summary Generation: Based on the requirement summary could be generated in numerous ways namely, extraction, topic lists, phrase concatenation and full sentence planning and generator.

2.3 Text Summarization using WordNet

Based on reading of *Generic Text Summarization using WordNet* [3].

Introduction: Selecting most representative sentences to generate a summary taking informativeness, relevance and conciseness into consideration. Unlike earlier approaches, this paper deals with semantics of the text over statistical aspects of the document.

WordNet: An online lexical reference system with English nouns, verbs, adjectives and adverbs organized in synsets (synonym sets). A WordNet graph is built and then extracts the most important and relevant topics from the document. Gets a global view of the document even before ranking the contents of the document.

Algorithm:

Sub-Graph Construction: Breaking into sentences, identifying collocations, removing stop words. Finding the portions of graph which are relevant to the text. First marks all the words which are directly present in the text and then do a breadth first search (upto a fixed depth) via the generalization edges to mark the reachable synsets.

Synset Ranking: Ranking the synsets based on their relevance to the document. Assigning higher rank for synsets with larger number of related words, this is based on the interpretation of WordNet synsets as representative units. Constructs a row vector R with entries representing the important of each node to the graph, and a square matrix A given by

$$A[i,j] = \begin{cases} 1/num\_predecessors(j), \text{ if } j \text{ is child of } i \\ 0, \text{ otherwise} \end{cases}$$

Inverse relationship in above equation can be interpreted as follows, if node j is connected to many other nodes then it's individual importance to node i is smaller. This as a whole captures the importance of a node (i.e synset) to the document by observing the edge relationships. The more related the node to the graph, the more important synset to the document and thus should be potentially included in the summary.

All values in row vector R are initialized to $1/\sqrt{n}$. Then iteratively apply the below transformation until it falls below a threshold value.

$$R_{new} = \frac{R_{old} * A}{|R_{old} * A|} \tag{2.2}$$

Sentence Selection: A matrix with relates sentences S_i in the input to synsets (ranked in above step), with sentences as rows and synsets as columns (matrix M) is given by,

$$M[S_i][sy_j] = \begin{cases} 0, \text{if } sy_j \text{ is not reachable from } S_i \\ R(sy_j), \text{otherwise} \end{cases}$$

where each row denotes the meaning captured by the sentence out of the whole text. Principal Component Analysis (PCA) is done to obtain the eigen vectors and sorts these sentences based on the eigen values. These sorted eigenvectors are now used to project all the sentences in the text and finally the sentences with highest projections are selected. This way sentences picked will be representing different semantic content because PCA converts possibly correlated variables to linearly uncorrelated variables.

2.4 Statistical Models

2.4.1 Latent Semantic Analysis

Overview: Building relationship between document and terms. A matrix is built with rows as individual words, columns as paragraphs and word count is put in as entry. Its a method for discovering hidden concepts in document data.

Singular Value Decomposition (SVD): Factorization of a complex matrix M into $U\Sigma V^*$, where U, V are unitary matrices (orthogonal in real domain) and Σ is diagonal matrix with entries known as singular values of M. Singular values are non-negative real numbers.

Applying SVD to the domain of LSA is decomposing a term-document matrix A into a term-concept matrix U, a singular matrix S and a concept-document matrix V as $A = USV^T$.

Given a term-document matrix A, we can obtain a document-document matrix $B = A^T A$ (correlation over terms) and a term-term matrix $C = AA^T$ (correlation over documents). And thus, $A = USV^T$

where U is the matrix of eigenvectors of B, V is the matrix of eigenvectors of C and Σ is a diagonal matrix with square root of eigenvalues of B as it's entries.

SVD reduces the number of rows but preserves the similarity structure among the columns, so effectively a dimensionality reduction technique. Query would be represented as the centroid of it's terms and cosine distance is used to rank the documents in relation to query.

The technique of LSA can also be appilied to the problem of summarization. For example in case of text summarization, we can use LSA to work on a term-sentence matrix. We then capture the interrelationships among the terms to semantically group terms or sentences. If a word combination is recurring in the document but salient, it is captured by the singular vectors and it's relative importance is given by the singular values.

2.4.2 Latent Semantic Indexing

Latent Semantic Indexing (LSI) is same as LSA but in the context it's application in Information Retrieval.

2.4.3 Latent Dirichlet Allocation

Based on reading of *Latent Dirichlet Allocation* [8].

Overview: Latent Semantic Indexing (LSI) and probabilistic latent semantic indexing (pLSI) are dimensionality reduction techniques. They work under the assumption of bag-of-words, i.e the order in the document is neglected for analysis purposes. It is equivalent to *exchangeability* for words within a document and also over the documents too, i.e the order of words in the document and the order of documents itself can be neglected.

To verify the claims of LSI and to analyse it's relative strengths and weaknesses, it is useful to develop a generative probabilistic model of text corpora and recover the aspects of the document by LSI. This is the motivation for latent dirichlet allocation (LDA).

Working: A word is discrete unit of data, document is a sequence of words and corpus is a collection of documents. Goal is to find a probabilistic model which not only assigns higher scores for contents of the document itself but also for similar documents.

LDA assumes a generative process for each document in the corpus,

- Number of words per document as chosen using a poisson distribution
- Choosing a topic mixture for document, using multinomial and dirichlet prior
- Generating the word by first picking up the topic (using above probability) and then using the topic to generate the word itself, using multinomial probability conditioned on the topic

LDA tries to backtrack the procedure of generation by a procedure called as Collapsed Gibbs Sampling.

Similar to LSA, LDA can also be used for summarization tasks. One of the recent works uses LDA to capture the events being covered by the documents and form the summaries with sentences representing these events [11]

Chapter 3

Sentiment Analysis

3.1 Detecting Domain Dedicated Polar Words

Based on reading of *Detecting Domain Dedicated Polar Words* [4].

Overview: *Universal Sentiment Lexicon* contains words with polarity invariant over domains, for example good, disappointing etc. But some words such as unpredictable, disastrous are positive in certain domains but negative in other domains. The goal is to develop a domain specific sentiment lexicon.

Algorithm: *Null hypothesis* states that given a neutral word, it has an equal chance of occuring in positive and negative documents. We perform Chi-squared test on the words in polar document and compare the output with threshold to reject null hypothesis (threshold is the minimum probability of accepting null hypothesis). Cleaning of corpus is also done to remove plot related information in movie domains.

Results: It experiments on a dataset consisting of 1000 positive and negative reviews each with threshold values as 1.07, 2.45, 3.84. Clean corpus indeed performed better. As the threshold value increases precision increased but recall decreased, thus leading to a fall in accuracy after some threshold.

3.2 Two Stage Sentiment Analyzer

Domain Sentiment Matters: A Two Stage Sentiment Analyzer [5].

Overview: Using *Domain Dedicated Polar Words* (DDPW) as a consise set for sentiment analysis in a domain. Some words have fluctuating polarity across domains but fixed in a given domain (*Chameleon words*). Regularity is consistency of use of a polar word in a particular domain. Both fluctuating polarity and regularity are missing from Universal Sentiment Lexicon (USL).

Algorithm: Chi-squared test:

$$X^2(W) = ((C_p - \mu)^2 + (C_n - \mu)^2)/\mu \tag{3.1}$$

μ is expected count or null hypothesis

This paper discusses about three classfication techniques, namely,

- *Neural Networks:* Uses complex non-linear hypothesis function for classification and currently considered as the state-of-art technique of the fast computational abilities to solve large neural networks.
- *Logistic Regression:* Sigmoid function as non-linear classifier, given by

$$Hypothesis(X) = \frac{1}{(1 + e^{(-Theta' * X)})} \tag{3.2}$$

  $Theta$ is a vector of weights associated to X

- *Support Vector Machine:* Classification is based on z score with boundaries as +1 and -1

$$z = Theta' * X \tag{3.3}$$

Results: Dataset consists of 1000 +ve and -ve movie reviews, 1000 +ve and -ve music instrument reviews and 700 +ve and -ve tourism reviews. Five fold cross validation is used and DDPW requires only few feature sets and also outperforms other methods such as unigrams, adjectives, top-adjectives, DDPW $\cup$ USL and USL in all the three classification techniques.

Chapter 4

Submodularity

4.1 Submodular Set Functions

A *set function* maps subsets of grount set V to real numbers, i.e $f : 2^V \rightarrow \Re$. A set function is said to sub-modular iff

$$\forall x \in V \backslash T \text{ and } \forall\, S, T \subseteq V,\ S \subseteq T \quad f(S \cup \{k\}) - f(S) \geq f(T \cup \{k\}) - f(T) \tag{4.1}$$

Submodular functions have the property of diminishing returns which finds it's direct application in Automatic Summarization. A function f is said to be normalized iff $f(\phi) = 0$ and f is said to be monotone if $\forall\, S, T\ s.t\ S \subseteq T,\ f(S) \leq f(T)$

4.2 A Class of Submodular Functions for Document Summarization

Based on reading of *A Class of Submodular Functions for Document Summarization* [6].

Overview: Designing a class of submodular functions for document summarization tasks. It also finds the hidden submodular optimization in currently established document summarization methods.

Problem: Summarization with Knapsack constraint

$$\text{Find } S^* \underset{S \subseteq V}{\operatorname{argmax}} F(S) \text{ subject to: } \sum_{i \in S} c_i \leq b \tag{4.2}$$

Montone Submodular Objectives Two importants criteria of a good summary are relevance and non-redundancy, both of these are incorporated into the objective as follows:

$$F(S) = L(S) + \lambda R(S) \tag{4.3}$$

where L measures the coverage and R measures diversity.

Coverage Function: L can be considered as the similarity of Summary S with the original document. L is also monotone submodular. There are numerous ways to define L, this paper proposes it to be:

$$L(S) = \sum_{i \in V} min\{C_i(S), \alpha C_i(V)\} \tag{4.4}$$

$$C_i(S) = \sum_{j \in S} w_{i,j} \tag{4.5}$$

$C_i(S)$ measures how similiar is summary to a sentence i. And having $\alpha < 1$ significantly improves performance than $\alpha = 1$.

Diversity Function: Instead of penalizing redundancy, they reward diversity by:

$$R(S) = \sum_{i=1}^{K} \sqrt{\sum_{j \in P_i \cap S} r_j} \tag{4.6}$$

where P_i denotes the partitions of ground set V (these could be predecided so as to cover more aspects of the document) and r_j donotes singleton reward function for j, in a way it estimates the importance of i to the summary. The square root function is responsible for the diminishing returns. $R(S)$ is different from $P(S)$ in the sense that it might include some outlier's which P might ignore.

Results: DUC-03 (Document Understanding Conference) is used as development set whereas DUC-04 is used as test data set. Cosine similarity is used as the similarity measure, where as reward function is as below,

$$r_j = \frac{1}{N} \sum_{i \in V} w_{i,j} \tag{4.7}$$

Using coverage function (L) alone outperformed the best system in DUC-04, while using reward function alone gave comparable performance in generating summary. The combined function outperforms the others significantly.

Thus submodularity is a natural fit for Summarization.

4.3 Multi-document Summarization via Budgeted Maximization

Based on readings of *Multi-document Summarization via Budgeted Maximization of Submodular Functions* [7].

Overview: Text Summarization problem as a submodular maximization under budget constraint.

$$\max_{S \subseteq V} \{f(S) : \sum_{i \in S} c_i \leq B\} \tag{4.8}$$

V is the ground set, S is summary and c_i is the non-negative cost of selecting i and B is the budget.

Though minimization of a submodular function is polytime, maximization is shown to be NP-hard. But fortunately it could be solved near-optimally. A famous result states that maximization under cardinality constraint (special case of budget constraint) can be solved by a greedy algorithm within a constant factor of 0.63 to the optimal. This paper specifically deals with Multi-document summarization.

Algorithm: Sequentially finding the unit k with largest ratio to objective function gain to scaled cost. If adding unit k increases the objective function without violating the budget constraint then it is selected.

$$k = \underset{l \in U}{\operatorname{argmax}} \frac{f(G \cup \{l\}) - f(G)}{(c_l)^r} \tag{4.9}$$

$$if \sum_{i \in G} c_i + c_k \leq B \; and \; f(G \cup \{k\}) - f(G) \geq 0, \quad G = G \cup \{k\} \tag{4.10}$$

These might result in unbounded approximation factor. Thus the following changes are made to the above algorithm,

- At the end, the result set G is compared to within-budget singleton with largest objective value.

$$v^* = \underset{v \in V, c_v \leq B}{\operatorname{argmax}} f(\{v\}) \tag{4.11}$$

- A scaling factor r is used on cost (as shown in above equation)

Theorem: For normalized monotone submodular functions, the above described algorithm ($r = 1$) has a constant factor approximation to optimal,

$$f(G_f) \geq (1 - e^{-\frac{1}{2}}) \; f(S^*) \tag{4.12}$$

In any case the solution is atleast as good as $0.39 \; f(S^*)$. Constant factor approximations are good because they are valid for all the instances of the problem.

Chapter 5

Working Implementation

5.1 Problem Description

Generating summary and extracting keywords for texts in Hindi.

5.2 Algorithm

Overview:

- Translating the Hindi text to English using "Translate Shell" [1]
- TextRank algorithm (Mihalcea et al. [1]) is used to both extract multi-word keywords and also generate extractive summary from the input document
- Retranslate the summary and keywords back to Hindi

5.3 Implementation

Both the summary generator and keyword extractor are implemented in Python.

5.3.1 Summary Generation

It consists of four stages namely,

Reading Data: Input data is parsed into sentences and words using NLTK sentence and word tokenizer respectively.

Building Graph: Each sentence is represented as a vertex in graph and similarity between the sentences is used as a weighted edge between two vertices. Similarity is computed as follows:

$$Similarity(S_i, S_j) = \frac{|\{w_k | w_k \in S_i \& w_k \in S_j\}|}{\log(|S_i|) + log(|S_j|)} \tag{5.1}$$

Iterate over Graph: The iteration equation is as given in equation 2.1 with threshold value as 0.00001

[1] Command line translator `www.soimort.org/translate-shell/`

Generate Summary: Sorts the sentences (vertices) based on the scores obtained in above step and outputs only the top sentences as summary. User get to choose the word limit of summary, is passed as an argument to program.

5.3.2 Keyword Extraction

Keyword extraction is important because it could be used for document browsing by providing a short summary of it's contents, this way it could find applications in improving information retrieval systems. Different stages in keyword extraction are reading data, POS tagging, syntactic filter, building graph, iteration over graph, generating keywords and collapsing keywords. Explaining the extra stages,

POS Tagging and Filtering: NLTK pos_tagger is used to tag the input words with their respective parts of speech. Only Nouns and Adjectives are considered as prospective keywords (this is based on observation).

Collapsing Keywords: After single-worded keywords are obtained, neighbouring keywords are combined into a multi-word keyword (or keyphrase). Number of keywords are limited to $1/3^{rd}$ the size of input.

5.4 Results

I have used a dataset of 14 Hindi texts (movie reviews) and compared the reference summaries with the program generated summaries. The average ROUGE-N score turned out be 0.33 and this would definitely improve by testing on a larger datasets and a greater number of reference summaries for each text. The results in detail are provided in a file `stats.txt`.

5.5 Inferences

We have chosen the approach of translating the text into English, do all the processing and at the end convert the document back to Hindi. But this approach might lead to sentences which are slightly different from input text (though that would not be the same with intermediate English text). This could be avoided by creating a mapping of input Hindi sentences to the translated sentences in English and after summarization, instead of retranslating we can directly use the reverse mapping to generate the output.

Translation seems to be the major influencing factor in the work, thus a better translator would lead to more relevant results. A better approach to this problem could be directly handle the Hindi text and trying to apply a modified version of TextRank directly to Hindi text without involving any translation. This would result a more realistic output similar to the case of TextRank on English text. The summarization technique could also be improved by considering the use of submodularity and WordNet.

Chapter 6

Conclusion

6.1 Scope of Work

This RnD work mainly focused on various techniques of Automatic Summarization. Learnt the applications of submodularity into the domain of Summarization and observed the direct resemblence of most of the existing techniques of summarization to submodularity. Studied through basics of Sentiment Analysis.

6.2 Recent Works

Some of the recent interesting works in the domain of Automatic Summarization are summarization of videos, automatic summarization of news articles in Android devices [12], automatic summarization of events from social media [13], summarization of bug reports [14].

Looking into above works, we can see the diverse applications of the techniques of Automatic Summarization.

6.3 Future works

Only the extractive approach to summarization has been explored in this work, abstractive summarization is an active area of research with techniques that would produce a more condensed and informative summaries. Using submodularity to extract summaries while retaining the sentiment of the text. We can also try to explore the need of summarization and re-design techniques for some of the real world requirements, in similar lines to few above stated works (social media, bug reports, video etc.,)

Bibliography

[1] Rada Mihalcea and Paul Tarau. 2004. *TextRank: Bringing Order into Texts.* Association of Computational Linguistics. `web.eecs.umich.edu/~mihalcea/papers/mihalcea.emnlp04.pdf`

[2] Eduard Hovy and Chin-Yew Lin. 1999. *Automated Text Summarization in SUMMARIST.* In Advances in Automatic Text Summarization, I. Mani and M. Maybury (editors). `www.isi.edu/natural-language/people/hovy/papers/98hovylin-summarist.pdf`

[3] Kedar Bellare, Anish Das Sarma, Atish Das Sarma, Navneet Loiwal, Vaibhav Mehta, Ganesh Ramakrishnan, Pushpak Bhattacharyya. 2004. *Generic Text Summarization using WordNet.* LREC. `http://lrec-conf.org/proceedings/lrec2004/pdf/342.pdf`

[4] Raksha Sharma and Pushpak Bhattacharyya. 2013. *Detecting Domain Dedicated Polar Words.* IJCNLP.

[5] Raksha Sharma and Pushpak Bhattacharyya. 2015. *Domain Sentiment Matters: A Two Stage Sentiment Analyzer.* ICON. `www.cse.iitb.ac.in/~pb/papers/icon15-domain-sentiment.pdf`

[6] Hui Lin and Jeff Bilmes. 2011. *A Class of Submodular Functions for Document Summarization.* In the Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies-Volume 1. `melodi.ee.washington.edu/~hlin/papers/lin-acl11-summ.pdf`

[7] Hui Lin and Jeff Bilmes. 2010. *Multi-document Summarization via Budgeted Maximization of Submodular Functions* Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics.

[8] David M. Blei, Andrew Y. Ng and Michael I. Jordan. 2003. *Latent Dirichlet Allocation.* Journal of Machine Learning Research 3 (2003) 993-1022. `http://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf`

[9] Josef Steinberger and Karel Jezek. 2004. *Using Latent Semantic Analysis in Text Summarization and Summary Evaluation* Proc. ISIM'04 93-100.

[10] Balamurali, A. R., Mitesh M. Khapra, and Pushpak Bhattacharyya. 2013. *Lost in translation: viability of machine translation for cross language sentiment analysis.* Computational Linguistics and Intelligent Text Processing. Springer Berlin Heidelberg, 2013. 38-49.

[11] Arora, Rachit, and Balaraman Ravindran. 2008. *Latent dirichlet allocation based multi-document summarization.* Proceedings of the second workshop on Analytics for noisy unstructured text data. ACM, 2008.

[12] Cabral, Luciano, et al. 2015. *Automatic Summarization of News Articles in Mobile Devices.* Fourteenth Mexican International Conference on Artificial Intelligence (MICAI). IEEE, 2015.

[13] Chua, Freddy Chong Tat, and Sitaram Asur. 2013. *Automatic Summarization of Events from Social Media.* ICWSM.

[14] Rastkar, Sarah, Gail C. Murphy, and Glen Murray. 2014. *Automatic summarization of bug reports.* Software Engineering, IEEE Transactions on 40.4 (2014): 366-380.